

GalacticApp Backend

Backend API del proyecto GalacticApp desarrollado para el Proyecto Final de UNaHur C2.

Descripción

Este es el servidor backend de la aplicación GalacticApp, construido con **Express.js** y **MongoDB**. Proporciona endpoints para la gestión de datos y comunicación con el frontend.

Requisitos Previos

Antes de comenzar, asegúrate de tener instalado:

- **Node.js** (v16 o superior)
- **npm** (se instala con Node.js)
- **MongoDB** (local o remoto)
- **Git**

Instalación

1. Clonar el repositorio:

```
git clone https://github.com/SebaBranda/Proyecto_Final_UNAHUR_C2-Backend.git
cd Proyecto_Final_UNAHUR_C2-Backend
```

2. Instalar dependencias:

```
npm install
```

3. Configurar variables de entorno:

- Crear un archivo **.env** en la raíz del proyecto
- Completar las variables requeridas (ver sección de Configuración)

Instalación con Docker

Requisitos Previos para Docker

- **Docker** (v20.10 o superior)
- **Docker Compose** (v2.0 o superior)

Pasos para ejecutar con Docker

1. Clonar el repositorio:

```
git clone https://github.com/SebaBranda/Proyecto_Final_UNAHUR_C2-Backend.git
cd Proyecto_Final_UNAHUR_C2-Backend
```

2. Crear archivo `.env` (opcional):

```
cp .env.example .env
```

Edita el archivo `.env` si deseas cambiar las credenciales de MongoDB u otros parámetros.

3. Levantar los servicios:

```
docker-compose up -d
```

Este comando:

- Construye la imagen Docker del backend
- Inicia un contenedor de MongoDB
- Inicia el servidor del backend
- Ambos servicios se conectan automáticamente

4. Verificar que está corriendo:

```
docker-compose ps
```

5. Ver logs de la aplicación:

```
docker-compose logs -f backend
```

6. Detener los servicios:

```
docker-compose down
```

Comandos útiles de Docker

```
# Ver logs de MongoDB
docker-compose logs mongodb
```

```
# Entrar a la shell del backend
docker-compose exec backend sh

# Reconstruir la imagen
docker-compose build --no-cache

# Eliminar volúmenes (CUIDADO: borra datos de MongoDB)
docker-compose down -v

# Ejecutar un comando específico
docker-compose exec backend npm run dev
```

Acceso a MongoDB desde Docker

- **Host:** mongodb
- **Puerto:** 27017
- **Usuario:** admin (por defecto)
- **Contraseña:** password (por defecto)
- **Base de datos:** galacticapp_db

Para conectarte desde fuera del contenedor:

```
mongodb://admin:password@localhost:27017/galacticapp_db
```

Despliegue en Producción con Docker

Para producción, utiliza:

```
docker-compose -f docker-compose.yml up -d
```

Asegúrate de:

- Cambiar las credenciales de MongoDB en `.env`
- Usar variables de entorno seguras
- Configurar HTTPS/SSL si es necesario
- Usar un reverse proxy como Nginx

Configuración

Variables de Entorno

Crear un archivo `.env` en la raíz del proyecto con las siguientes variables:

```
# Puerto del servidor
PORT=3000
```

```
# URI de conexión a MongoDB
MONGO_URI=mongodb://localhost:27017/galacticapp_db

# (Agregar otras variables según sea necesario)
```

Conexión a MongoDB

La aplicación se conecta automáticamente a MongoDB al iniciar. La URI por defecto es:

```
mongodb://localhost:27017/galacticapp_db
```

Para usar una base de datos remota, actualiza **MONGO_URI** en el archivo **.env**.

Scripts Disponibles

```
# Iniciar servidor en modo desarrollo (con nodemon)
npm run dev

# Iniciar servidor en producción
npm start
```

Estructura del Proyecto

```
.
├── index.js           # Archivo principal
├── package.json       # Dependencias y scripts
├── .env               # Variables de entorno (no versionado)
├── .gitignore         # Archivos ignorados por git
└── README.md         # Este archivo
```

Dependencias Principales

- **express** (^5.2.1): Framework web para Node.js
- **mongoose** (^9.9.4): ODM para MongoDB
- **cors** (^2.8.6): Middleware para CORS
- **dotenv** (^17.4.2): Gestor de variables de entorno
- **nodemon** (^3.1.14): Monitor de cambios en desarrollo

Endpoints Base

Salud de la API

```
GET http://localhost:3000/
```

Respuesta:

```
API de GalacticApp funcionando correctamente
```

Configuración de CORS

CORS está habilitado en todas las rutas para permitir solicitudes desde el frontend. Configurable en [index.js](#).

Middlewares

- **CORS:** Permite solicitudes desde diferentes orígenes
- **Express JSON:** Parsea automáticamente JSON en los requests

Notas Importantes

- El servidor escucha por defecto en el puerto **3000**
- MongoDB debe estar ejecutándose localmente o accesible remotamente
- Asegúrate de que el archivo [.env](#) no se versionea (incluido en [.gitignore](#))
- Usa `npm run dev` durante el desarrollo para recargas automáticas

Troubleshooting

Error: "Cannot connect to MongoDB"

- Verifica que MongoDB esté corriendo
- Comprueba la URI en la variable [MONGO_URI](#)
- Revisa tu conexión de red si usas MongoDB remoto

Error: "Port already in use"

- Cambia el puerto en la variable [PORT](#) del [.env](#)
- O termina el proceso que está usando el puerto 3000

Contacto y Soporte

Para reportar issues o sugerencias, utiliza [GitHub Issues](#).

Licencia

ISC - Ver archivo package.json para más detalles